

Reflex — Event-Driven Delivery Management

Small Kenyan Retailers - Real-Time Delivery
Tracking

Presented by: Sir Nchoe Leshan
Date: September 5, 2026

Small Retailers Manage Deliveries via WhatsApp — No Tracking

- . Retailers coordinate deliveries over WhatsApp and phone calls
- . No record of who's assigned to which delivery
- . No status visibility – "Where is the order?"
- . No proof of delivery
- . Result: Lost orders, frustrated customers, wasted time

E.g., "A retailer loses an order because they forgot which rider took it."

Reflex – One System for All Delivery Tracking

- . Retailer logs a delivery request
- . Dispatcher assigns it to a rider
- . Rider updates the status in real-time
- . Everyone knows where every delivery stands

Transparency · Accountability · Real-Time Visibility

Event-Driven Architecture: Queue + Webhooks + SSE

Retailer — (Requests)

Queue — (Pending)

Webhook — (Completed)

UI Shows — "Pending"

Assigns — Dispatcher

Delivers — Rider

Queue — Webhook — SSE = Real-Time, Event-Driven System

Attendee Status: Pending — Checked In

Attendee {

id: string (e.g., "ATT-001")

name: string

checked_in: boolean (Final state)

pending: boolean (Waiting for webhook)

checked_in_at: timestamp

}

Pending State = We Know the Request is in Progress.

Checked In State = Confirmed by the Webhook.

Scan — Queue — Webhook — Complete

1. Retailer scans the order.

- Publishes to queue.
- UI shows "Pending".

2. Dispatcher sees the request.

- Assigns to a rider.

3. Rider completes the delivery.

- Calls the webhook.

4. System updates the status.

- SSE pushes real-time update.
- UI shows "Checked In" instantly.

Retailer, Dispatcher, Rider – Each Has a Role

,

Retailer	Dispatcher	Rider
Logs delivery requests	Sees open requests	Sees assigned deliveries
Tracks status in real-time	Assigns to a rider	Updates status(picked up, delivered)

Made Conscious Trade-offs — Here's Why

1. In-Memory Storage vs. Database

- Accepted because: Prototype, need to ship fast
- Cost: Data lost on restart
- Future: Swap with SQLite

2. SSE vs. WebSockets

- Accepted because: Simpler, server–client only
- Cost: No bi-directional communication
- Future: Switch if needed

3. In-Memory Queue vs. RabbitMQ

- Accepted because: No external dependencies
- Cost: Queue not persistent
- Future: Use Redis for production

Handle Edge Cases Gracefully

1. Duplicate Scans

- Second scan rejected with 409 Conflict
- `scanned\_tickets` set prevents duplicates

2. Out-of-Order Webhooks

- Tracking is independent of callback order
- Duplicate protection still holds

3. API Failures

- 10% simulated failure rate
- Clear error messages shown to user

What's Next for Reflex

1. Persistent Storage

- SQLite to survive server restarts

2. Retry Logic

- Exponential backoff for failed webhooks

3. QR Scanner Integration

- Real QR code scanning via webcam

4. Admin Dashboard

- Real-time metrics and analytics

5. Containerization

- Docker + docker-compose for easy deployment